

CSE430 - Software Testing and Quality Assurance

Lab Report: Pytest Unit Testing (Homework 1, 2, 3)

Name: Akhlak Hossain

Student ID: 2022-3-60-057

August 6, 2026

1 Homework 1: Password Strength Checker

1.1 Code Snippets

1.1.1 Base Code (homework.py)

```
1 class PasswordChecker:
2     def is_strong(self, password):
3         if len(password) < 8:
4             return False
5         if not any(char.isdigit() for char in password):
6             return False
7         if not any(char.isupper() for char in password):
8             return False
9         return True
```

1.1.2 Test Code (test_homework.py)

```
1 @pytest.fixture
2 def checker_fixture():
3     return PasswordChecker()
4
5 @pytest.mark.parametrize(
6     "password, expected",
7     [
8         ("StrongPass1", True),           # Strong password
9         ("Short1A", False),              # Too short
10        ("nodigitpassA", False),         # No digit
11        ("nouppercase1", False),         # No uppercase
12        ("", False),                     # Empty string
13    ]
14 )
15 def test_password_strength(checker_fixture, password, expected):
16     assert checker_fixture.is_strong(password) == expected
```

1.2 Test-case Table

Method	Inputs (password)	Expected Output	Result
is_strong	"StrongPass1"	True	PASSED
is_strong	"Short1A"	False	PASSED
is_strong	"nodigitpassA"	False	PASSED
is_strong	"nouppercase1"	False	PASSED
is_strong	""	False	PASSED

1.3 Reflection

When writing the `PasswordChecker` class, I found that the empty string edge case was technically covered by the minimum length requirement, but the hardest rule to test correctly was the inclusion of digits and uppercase letters. It required leveraging Python's `any()` function combined with list comprehensions to efficiently iterate through the string without writing overly verbose loops. Ensuring the fixture was injected properly into the parametrized test

demonstrated how efficiently Pytest can run multiple scenarios over a fresh instance without duplicating setup code.

2 Homework 2: Movie Ticket Booking

2.1 Code Snippets

2.1.1 Base Code (homework.py)

```
1 class Cinema:
2     def __init__(self, total_seats):
3         self.total_seats = total_seats
4         self.booked_seats = 0
5
6     def book_seats(self, n):
7         if n > self.available_seats():
8             raise ValueError("Not enough seats available.")
9         self.booked_seats += n
10
11     def available_seats(self):
12         return self.total_seats - self.booked_seats
13
14     def is_sold_out(self):
15         return self.available_seats() == 0
```

2.1.2 Test Code (test_homework.py)

```
1 def test_normal_booking():
2     cinema = Cinema(10)
3     cinema.book_seats(4)
4     assert cinema.available_seats() == 6
5     assert cinema.is_sold_out() == False
6
7 def test_exact_sellout():
8     cinema = Cinema(5)
9     cinema.book_seats(5)
10    assert cinema.available_seats() == 0
11    assert cinema.is_sold_out() == True
12
13 def test_overbooking_attempt():
14     cinema = Cinema(10)
15     with pytest.raises(ValueError) as excinfo:
16         cinema.book_seats(12)
17     assert str(excinfo.value) == "Not enough seats available."
18
19 def test_booking_zero_seats():
20     cinema = Cinema(10)
21     cinema.book_seats(0)
22     assert cinema.available_seats() == 10
23     assert cinema.is_sold_out() == False
24
25 def test_sequential_bookings():
26     cinema = Cinema(15)
27     cinema.book_seats(5)
28     cinema.book_seats(5)
29     assert cinema.available_seats() == 5
30     assert cinema.is_sold_out() == False
```

2.2 Test-case Table

Method (Scenario)	Inputs	Expected Output	Result
book_seats (Normal)	total=10, n=4	available_seats()=6, is_sold_out()=False	PASSED
book_seats (Sellout)	total=5, n=5	available_seats()=0, is_sold_out()=True	PASSED
book_seats (Overbook)	total=10, n=12	ValueError	PASSED
book_seats (Zero)	total=10, n=0	available_seats()=10	PASSED
book_seats (Sequential)	total=15, n=5,5	available_seats()=5	PASSED

3 Homework 3: Inventory Restock Tracker

3.1 Code Snippets

3.1.1 Base Code (homework.py)

```
1 class InventoryItem:
2     def __init__(self, quantity, reorder_level):
3         self.quantity = quantity
4         self.reorder_level = reorder_level
5
6     def restock(self, amount):
7         self.quantity += amount
8
9     def sell(self, amount):
10        if amount > self.quantity:
11            raise ValueError("Amount exceeds current quantity.")
12        self.quantity -= amount
13
14    def needs_reorder(self):
15        return self.quantity <= self.reorder_level
```

3.1.2 Test Code (test_homework.py)

```
1 def test_sell_exceeds_quantity_raises_value_error():
2     item = InventoryItem(quantity=10, reorder_level=5)
3     with pytest.raises(ValueError) as excinfo:
4         item.sell(15)
5     assert str(excinfo.value) == "Amount exceeds current quantity."
6
7 def test_needs_reorder_returns_true_at_or_below_level():
8     item = InventoryItem(quantity=6, reorder_level=5)
9     assert item.needs_reorder() == False
10    item.sell(1) # drops exactly to reorder level
11    assert item.needs_reorder() == True
12    item.sell(2) # drops below reorder level
13    assert item.needs_reorder() == True
14
15 @pytest.mark.parametrize(
16     "initial_qty, reorder_lvl, restock_amt, sell_amt, expected_qty",
17     [
18         (10, 5, 5, 2, 13),
19         (0, 5, 20, 20, 0),
20         (50, 10, 0, 10, 40)
21     ]
22 )
23 def test_restock_and_sell_combinations_update_quantity(initial_qty,
24     reorder_lvl, restock_amt, sell_amt, expected_qty):
25     item = InventoryItem(initial_qty, reorder_lvl)
26     item.restock(restock_amt)
27     item.sell(sell_amt)
28     assert item.quantity == expected_qty
```

Method	Inputs (qty, lvl, restk, sell)	Expected Output	Result
sell (ValueError)	10, 5, restk=0, sell=15	ValueError	PASSED
needs_reorder	6, 5, restk=0, sell=1	True (qty=5)	PASSED
restock & sell	10, 5, restk=5, sell=2	quantity = 13	PASSED
restock & sell	0, 5, restk=20, sell=20	quantity = 0	PASSED
restock & sell	50, 10, restk=0, sell=10	quantity = 40	PASSED

3.2 Test-case Table

3.3 Observation on Edge Cases

While testing the boundary condition for `needs_reorder()`, I observed a significant edge case when the quantity drops to *exactly* the `reorder_level`. Testing strictly for $\leq$ instead of $<$ is crucial here. If an item has a reorder level of 5 and drops to exactly 5, the test `test_needs_reorder_returns_true_at_or_below_level` ensures that a restock alert is triggered immediately rather than waiting for the inventory to fall to 4, preventing potential stockouts.

4 Screenshot of Test Runs

Note: Below is the verbose output generated by executing the tests via terminal.

```
$ pytest -v
===== test session starts =====
platform win32 -- Python 3.10.x, pytest-x.x.x, pluggy-x.x.x
rootdir: C:\Users\Student\CalculatorApp
collected 13 items

test_homework.py::test_password_strength[StrongPass1-True] PASSED
test_homework.py::test_password_strength[Short1A-False] PASSED
test_homework.py::test_password_strength[nodigitpassA-False] PASSED
test_homework.py::test_password_strength[nouppercase1-False] PASSED
test_homework.py::test_password_strength[-False] PASSED
test_homework.py::test_normal_booking PASSED
test_homework.py::test_exact_sellout PASSED
test_homework.py::test_overbooking_attempt PASSED
test_homework.py::test_booking_zero_seats PASSED
test_homework.py::test_sequential_bookings PASSED
test_homework.py::test_sell_exceeds_quantity_raises_value_error PASSED
test_homework.py::test_needs_reorder_returns_true_at_or_below_level PASSED
test_homework.py::test_restock_and_sell_combinations_update_quantity[10-5-5-2-13] PASSED
test_homework.py::test_restock_and_sell_combinations_update_quantity[0-5-20-20-0] PASSED
test_homework.py::test_restock_and_sell_combinations_update_quantity[50-10-0-10-40] PASSED

===== 13 passed in 0.12s =====
```